

spinalSynth: Testing Specification

Table of Contents

1. Unit Tests

- 1.1 Attenuator Unit Test (`AttenuatorSim`)
- 1.2 Register Bank Unit Test (`RegisterBankSim`)
- 1.3 UART Protocol Decoder Unit Test (`UartProtocolDecoderSim`)
- 1.4 Timing Generator Unit Test (`TimingSim`)
- 1.5 Waveform Generator Unit Test (`WaveformSim`)
- 1.6 I2S Transmitter Unit Test (`I2STransmitterSim`)
- 1.7 Envelope Control Unit Test (`EnvelopeCtrlSim`)
- 1.8 Envelope Phase Accumulator Unit Test (`EnvelopeAccumulatorSim`)
- 1.9 Envelope Waveshaper Unit Test (`EnvelopeShaperSim`)

2. Integration Tests

- 2.1 Complete System Integration Test (`SynthSim`)
- 2.2 Complete Envelope Generator Integration Test (`EnvelopeGeneratorSim`)

3. Appendix: SpinalSim Best Practices for Reset Verification

- A.1 The Verilator Combinational Loop Problem (FSM Gating)
- A.2 Thread-Safe Reset Verification via `forkStimulus` Startup
- A.3 State Transition Synchronization (Delta-Cycle Waiting)

1. Unit Tests

This chapter contains the specifications for testing individual, isolated hardware modules.

1.1 Attenuator Unit Test (AttenuatorSim)

Purpose

Verifies the custom DSP volume attenuator module (Attenuator) across default 8-bit and parameterized 10-bit configurations for mathematical precision, pipeline latency, and reset stability.

Simulated Environment

- **Component Under Test:** Attenuator
- **Clock Domain:** 24 MHz synchronous master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.sampleIn`: Flow[SInt] (16 bits)
- `io.volume`: UInt (volumeWidth bits)
- `io.sampleOut`: Flow[SInt] (16 bits)

Test Cases

1.1.1 Reset Stability

- **Action:** Assert reset high for 5 clock cycles while driving random values on `io.sampleIn.payload` and `io.volume`.
- **Assertion:** Verify that `io.sampleOut.payload` is strictly held at 0 and `io.sampleOut.valid` is strictly False.

1.1.2 Mathematical Scaling

- **Action:** Pulse `io.sampleIn.valid` high for 1 clock cycle with a specific payload and volume, then return it to False. Wait 1 clock cycle for the pipeline register.
- **Assertion:** Verify that `io.sampleOut.valid` is True during the second cycle and its payload matches the expected fractional attenuation exactly:

expected = (sampleIn.payload * volume) / (2^volumeWidth)

- **Test Vectors (8-bit Default):**

Input Sample	Volume	Expected Output	Notes
20000	255	19921	Full scale fractional (19921.8 truncated)
20000	128	10000	Half scale (Exact)
-20000	64	-5000	Quarter scale negative
-32768	0	0	Silent (Mute)

1.1.3 Cycle-Accurate Latency & Pipelining

- **Action:** Pulse `io.sampleIn.valid` back-to-back for 3 consecutive clock cycles with distinct samples:

- Cycle 1: Sample = 10000, Volume = 255
- Cycle 2: Sample = 20000, Volume = 128
- Cycle 3: Sample = -10000, Volume = 64
- **Assertion:** Verify that:
 - Cycle 1: `sampleOut.valid` is `False`.
 - Cycle 2: `sampleOut.valid` is `True`, payload = 9960.
 - Cycle 3: `sampleOut.valid` is `True`, payload = 10000.
 - Cycle 4: `sampleOut.valid` is `True`, payload = -2500.
 - Cycle 5: `sampleOut.valid` is `False`.
 - This confirms the 1-cycle pipeline throughput operates continuously without stalls or register leakage.

1.1.4 Parameterized 10-bit Configuration

- **Action:** Instantiate the unit under test with `volumewidth = 10`.
- **Test Vectors:**

Input Sample	Volume	Expected Output	Notes
20000	1023	19980	Full scale fractional (20000 * 1023 / 1024)
20000	512	10000	Half scale (Exact)
-20000	256	-5000	Quarter scale negative
-32768	0	0	Silent (Mute)

- **Pipelined Throughput Test:**
 - Cycle 1: Sample = 10000, Volume = 1023
 - Cycle 2: Sample = 20000, Volume = 512
 - *Assertion:* `sampleOut.valid` is `True`, payload = 9990.
 - Cycle 3: Sample = -10000, Volume = 256
 - *Assertion:* `sampleOut.valid` is `True`, payload = 10000.
 - Cycle 4: Idle (`sampleIn.valid` is `False`)
 - *Assertion:* `sampleOut.valid` is `True`, payload = -2500.
 - Cycle 5: Idle
 - *Assertion:* `sampleOut.valid` is `False`.
 - This confirms scaling math updates automatically based on `volumewidth`.

1.2 Register Bank Unit Test (RegisterBankSim)

Purpose

Verifies the parameter storage register bank module (`RegisterBank`) for reset defaults, single-byte register updates, and atomic 24-bit frequency word commitment.

Simulated Environment

- **Component Under Test:** `RegisterBank`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.regWrite`: `Flow[RegisterWrite]` (containing 8-bit address and 8-bit data)
- `io.config`: `OscillatorConfig` (output bundle containing `freqWord`, `waveSelect`, `pwmWidth`, `volume`)
- `io.envConfig`: `EnvelopeConfig` (output bundle containing `ctrl`, `attack`, `decay`, `sustain`, `release`, `syncCtrl`, `phaseOffset`)

Test Cases

1.2.1 Reset Defaults

- **Action:** Start simulation with reset asserted.
- **Assertion:** Verify that all output configuration fields in both `io.config` and `io.envConfig` are held strictly at 0.

1.2.2 Single-Byte Direct Updates

- **Action:** Write individually to non-atomic registers using the `io.regWrite` port:
 - Write 0x03 (address 0x03 - WAVE_SEL)
 - Write 0xA5 (address 0x04 - PWM_WIDTH)
 - Write 0x7F (address 0x05 - VOLUME)
- **Assertion:** Verify that the corresponding output fields (`config.waveSelect`, `config.pwmWidth`, and `config.volume`) are updated to the written values on the next clock cycle.

1.2.3 Atomic 24-Bit Frequency Commitment

- **Action:** Perform a sequential write sequence to verify shadow staging and atomic trigger commitment:
 1. Write 0x55 to 0x00 (FREQ_LOW).
 - *Assertion:* Verify that `config.freqWord` remains unchanged (stages in shadow register).
 2. Write 0xAA to 0x01 (FREQ_MID).
 - *Assertion:* Verify that `config.freqWord` remains unchanged (stages in shadow register).
 3. Write 0x0C to 0x02 (FREQ_HIGH).
 - *Assertion:* Verify that on the next clock cycle, the active output `config.freqWord` updates atomically to 0x0CAA55 (830037 in decimal) all at once.

1.2.4 Envelope Parameter Updates

- **Action:** Write individually to all seven envelope registers using `io.regWrite`:
 - Write 0x15 to 0x40 (ENV_CTRL)
 - Write 0x0A to 0x41 (ENV_ATTACK)
 - Write 0x1F to 0x42 (ENV_DECAY)
 - Write 0x80 to 0x43 (ENV_SUSTAIN)
 - Write 0x2C to 0x44 (ENV_RELEASE)
 - Write 0x05 to 0x45 (ENV_SYNC_CTRL)
 - Write 0xFF to 0x46 (ENV_PHASE_OFFSET)
- **Assertion:** Verify that the corresponding output fields in `io.envConfig` (`ctrl`, `attack`, `decay`, `sustain`, `release`, `syncCtrl`, `phaseOffset`) are updated to the written values on the next clock cycle.

1.2.5 Address Crosstalk & Channel Isolation

- **Action:** Perform crosstalk validation across register regions:
 1. Write arbitrary values to all oscillator registers (0x00 through 0x05).
 - *Assertion:* Verify that all envelope fields in `io.envConfig` remain completely unchanged.
 2. Write arbitrary values to all envelope registers (0x40 through 0x46).
 - *Assertion:* Verify that all oscillator fields in `io.config` remain completely unchanged.

1.3 UART Protocol Decoder Unit Test (UartProtocolDecoderSim)

Purpose

Verifies the FSM protocol parser (`UartProtocolDecoder`) for reset safety, successful command framing, byte-spacing delay tolerance, and command-valid bounds.

Simulated Environment

- **Component Under Test:** `UartProtocolDecoder`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.rxByte: Flow[Bits]` (input stream representing incoming serial bytes)
- `io.regWrite: Flow[RegisterWrite]` (output transaction flow)

Test Cases

1.3.1 Reset Safety

- **Action:** Assert reset while pulsing `rxByte.valid` with random bytes.
- **Assertion:** Verify that `io.regWrite.valid` remains strictly `False`.

1.3.2 Valid Command Framing (3-Byte Stream)

- **Action:** Push three bytes back-to-back:
 - Byte 1: `0x01` (`WriteRegister` command)
 - Byte 2: `0x02` (`FREQ_HIGH` address)
 - Byte 3: `0xAB` (Register data payload)
- **Assertion:** Verify that:
 - `regWrite.valid` is `False` after Byte 1 and Byte 2.
 - `regWrite.valid` is `True` in the exact cycle Byte 3 is pushed, and its payload contains `address = 0x02` and `data = 0xAB`.
 - `regWrite.valid` drops back to `False` in the following cycle.

1.3.3 Byte-Spacing Delay Tolerance

- **Action:** Push a 3-byte transaction with arbitrary timing spacing to simulate slow UART transmissions:
 - Push Byte 1 (`0x01`), then wait 25 clock cycles.
 - Push Byte 2 (`0x05 - VOLUME`), then wait 50 clock cycles.
 - Push Byte 3 (`0x7F`), then wait 1 clock cycle.
 - **Assertion:** Verify that the FSM maintains state synchronization, and atomically asserts `regWrite.valid = True` with `address = 0x05` and `data = 0x7F` exactly when Byte 3 is pushed.
-

1.4 Timing Generator Unit Test (`TimingSim`)

Purpose

Verifies the master clock divider module (`TimingGenerator`) for reset safety, cycle-accurate tick intervals (`480\text{ kHz}`) phase ticks and `48\text{ kHz}` sample ticks, and strict synchronous alignment between clock domains.

Simulated Environment

- **Component Under Test:** `TimingGenerator`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.phaseTick: Bool` (output divider pulse every 50 master clock cycles)

- `io.sampleTick`: Bool (output divider pulse every 500 master clock cycles)

Test Cases

1.4.1 Reset Stability

- **Action:** Assert active-high reset for 20 clock cycles.
- **Assertion:** Verify that both `phaseTick` and `sampleTick` remain strictly False.

1.4.2 Interval Precision & Long-Term Stability

- **Action:** Run the simulation and measure consecutive tick intervals after deasserting reset:
 - Track 10 consecutive `phaseTick` events.
 - Track 5 consecutive `sampleTick` events.
- **Assertion:** Assert that every `phaseTick` interval is exactly 50 clock cycles and every `sampleTick` interval is exactly 500 clock cycles, ensuring zero clock drift.

1.4.3 Synchronous Domain Alignment

- **Action:** Monitor outputs over 5 full sample cycles (2500 clock cycles).
- **Assertion:** Verify that *every single time* `sampleTick` is asserted (True), `phaseTick` is also synchronously asserted (True).

1.5 Waveform Generator Unit Test (WaveformSim)

Purpose

Verifies the digital oscillators core module (Generators) for mathematical wave formatting precision (Sawtooth, Square, Triangle) and pulse-width comparator thresholds (PWM) across boundary phases.

Simulated Environment

- **Component Under Test:** Generators
- **Clock Domain:** None (Combinational Verification).

Input Stimulus & Signals

- `io.phase`: UInt (24 bits)
- `io.pwmWidth`: UInt (8 bits)
- `io.waves`: Waveforms (output bundle containing saw, square, pwm, tri)

Test Cases

1.5.1 Peak & Zero-Crossing Waveform Math

- **Action:** Drive specific static phase angles and assert output bounds:
 - **Phase 0x000000 (Start):** Assert saw = -32768, square = -32768, tri = -32768.
 - **Phase 0x400000 (1/4 Cycle):** Assert saw = -16384, square = -32768, tri = 0.
 - **Phase 0x7FFFFFF (Pre-Half Transition):** Assert saw = -1, square = -32768, tri = 32767.
 - **Phase 0x800000 (Half Cycle / Toggle):** Assert saw = 0, square = 32767, tri = 32767.
 - **Phase 0xC00000 (3/4 Cycle):** Assert saw = 16384, square = 32767, tri = -1.
 - **Phase 0xFFFFFFFF (Wrap Boundary):** Assert saw = 32767, square = 32767, tri = -32768.

1.5.2 PWM Comparator Threshold Logic

- **Action:** Set pulse duty configurations and verify switching thresholds:

- Set `pwmWidth = 0x80` (50%): Verify `pwm = 32767` at phase `0x7FFFFFF` and `pwm = -32768` at phase `0x800000`.
- Set `pwmWidth = 0x40` (25%): Verify `pwm = 32767` at phase `0x3FFFFFF` and `pwm = -32768` at phase `0x400000`.

1.6 I2S Transmitter Unit Test (I2STransmitterSim)

Purpose

Verifies the I²S serial audio transmitter module (I2STransmitter) for startup safety, idle low power states, cycle-accurate timing patterns (according to the modulation table), and correct parallel-to-serial data conversion.

Simulated Environment

- **Component Under Test:** I2STransmitter
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.sampleIn`: Flow[SInt] (16 bits, incoming parallel samples)
- `io.bclk`: Bool (output continuous bit clock)
- `io.lrcclk`: Bool (output left/right word select channel line)
- `io.sdata`: Bool (output serialized data line)

Test Cases

1.6.1 Reset Stability & Idle Power State

- **Action:** Assert reset for 20 clock cycles, deassert, and run for 20 more cycles without pulsing `sampleIn.valid`.
- **Assertion:** Verify that throughout this duration:
 - `bclk` is held strictly `False`.
 - `lrcclk` is held strictly `True` (standard passive high line state).
 - `sdata` is held strictly `False`.

1.6.2 Serial Word Bitstream Precision

- **Action:** Pulse `sampleIn.valid = True` for 1 cycle loading sample `0xA5A5` (binary `1010010110100101`).
- **Assertion:** Align with the start of the serialization frame (`lrcclk` goes `False` for Left channel), and sample `sdata` at each bit's middle interval (when `bclk` is `True`):
 - Verify that the stream of 16 sequential bits matches the MSB-first bits of `0xA5A5` perfectly.

1.6.3 Timing Pattern & Frame Duration

- **Action:** Wait for `lrcclk` to toggle, then measure the individual bit clock intervals and the full Left-to-Right frame duration.
- **Assertion:** Verify that:
 - The first 8 bit intervals match the cycle-accurate modulation pattern: 16, 16, 15, 16, 16, 15, 16, 15 clock cycles.
 - The full Left/Right stereo frame completes in exactly 500 clock cycles (48 kHz sampling rate).

1.7 Envelope Control Unit Test (EnvelopeCtrlSim)

Purpose

Verifies the ADSR state machine FSM built using SpinalHDL's `StateMachine` library, sync trigger propagation, playback direction logic, and compile-time logarithmic parameter-to-increment lookup ROM.

Simulated Environment

- **Component Under Test:** `EnvelopeCtrl`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.syncIn`: `Bool` (Hard/Soft sync line)
- `io.midiClock`: `Bool` (MIDI clock division reference)
- `io.config`: `EnvelopeConfig` (Input registers)
- `io.segmentDone`: `Bool` (Accumulator target completion)
- `io.resetAccum`: `Bool` (Output to reset accumulator register)
- `io.runAccum`: `Bool` (Output to run accumulator register)
- `io.accumDir`: `Bool` (Output direction)
- `io.phaseInc`: `UInt` (22 bits, phase step value)
- `io.curveSelect`: `UInt` (2 bits, curve selector)
- `io.activeStage`: `UInt` (3 bits, state stage output)

Test Cases

1.7.1 Reset Stability

- **Action:** Assert active-high reset for 5 clock cycles while driving random stimulus values on `io.syncIn`, `io.midiClock`, `io.config`, and `io.segmentDone`.
- **Assertion:** Verify that all control outputs are strictly held at their idle states: `io.resetAccum = False`, `io.runAccum = False`, `io.accumDir = False`, `io.phaseInc = 0`, `io.curveSelect = 0`, and `io.activeStage = 0`.

1.7.2 Normal ADSR State Transitions

- **Action:** Configure standard parameters and toggle the Gate register bit `config.ctrl[1]`:
 1. Toggle Gate Low -> High while FSM is in IDLE.
 - *Assertion:* FSM transitions to ATTACK (`activeStage = 1`), asserts `resetAccum = True` and `runAccum = True` for 1 cycle.
 2. Pulse `segmentDone` High for 1 cycle.
 - *Assertion:* FSM transitions to DECAY (`activeStage = 2`), asserts `resetAccum = True` for 1 cycle.
 3. Pulse `segmentDone` High for 1 cycle.
 - *Assertion:* FSM transitions to SUSTAIN (`activeStage = 3`), disables phase tracking (`runAccum = False`).
 4. Toggle Gate High -> Low.
 - *Assertion:* FSM transitions to RELEASE (`activeStage = 4`), asserts `resetAccum = True` and `runAccum = True` for 1 cycle.
 5. Pulse `segmentDone` High for 1 cycle.
 - *Assertion:* FSM transitions back to IDLE (`activeStage = 0`), clears `runAccum = False`.

1.7.3 Gate Interruption & Re-triggering

- **Action:** Interrupt active phases with unexpected Gate toggles:
 - Trigger Gate ON, transition to ATTACK, then toggle Gate OFF halfway through.
 - *Assertion:* FSM transitions instantly to RELEASE (`activeStage = 4`).
 - Trigger Gate OFF, transition to RELEASE, then toggle Gate ON halfway through.
 - *Assertion:* FSM transitions instantly to ATTACK (`activeStage = 1`) and resets accumulator (`resetAccum = True`).

1.7.4 Looping (LFO) Mode

- **Action:** Set Loop Enable register `config.ctrl[2] = True`. Trigger Gate ON. Allow FSM to transition through ATTACK to DECAY. Pulse `segmentDone` High.
- **Assertion:** Verify that FSM transitions instantly from DECAY back to ATTACK (`activeStage = 1`) instead of going to SUSTAIN, resetting the accumulator cleanly.

1.7.5 Logarithmic Increment ROM Mapping

- **Action:** Write specific values to the `attack` register and read `phaseInc` outputs:
 - Write `ENV_ATTACK = 0` (minimum rate parameter).
 - *Assertion:* Verify `phaseInc` is exactly 357914 (0x05761A), yielding a 0.5 ms transient.
 - Write `ENV_ATTACK = 255` (maximum rate parameter).
 - *Assertion:* Verify `phaseInc` is exactly 6 (0x000006), yielding a 30.0 s transient.

1.8 Envelope Phase Accumulator Unit Test (EnvelopeAccumulatorSim)

Purpose

Verifies the 32-bit register accumulator, phase counting direction, split output mapping, and segment overflow boundaries.

Simulated Environment

- **Component Under Test:** EnvelopeAccumulator
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.resetAccum`: Bool
- `io.runAccum`: Bool
- `io.accumDir`: Bool (0 = Forward, 1 = Reverse)
- `io.phaseInc`: UInt (22 bits)
- `io.segmentDone`: Bool (Output complete)
- `io.baseIndex`: UInt (8 bits, output address)
- `io.fraction`: UInt (2 bits, output fraction)

Test Cases

1.8.1 Reset Defaults

- **Action:** Assert active-high reset for 5 clock cycles while driving arbitrary inputs on `runAccum = True`, `accumDir = True`, `activeStage = 0`, `sustainLevel = 0`, and `phaseInc = 0x200000` (valid 22-bit value).
- **Assertion:** Verify that the internal phase register remains strictly at 0, outputting `baseIndex = 0`, `fraction = 0`, and `segmentDone = False`.

1.8.2 Phase Accumulation & Splitting Precision

- **Action:** Drive `phaseInc = 0x200000` (2^{21}). Enable the accumulator and set `activeStage = 1` (Attack).
- **Assertion:** Verify the splitting output precision:
 - Cycle 1: `baseIndex = 0`, `fraction = 0`.
 - Cycle 2: `baseIndex = 0`, `fraction = 1`.
 - Cycle 3: `baseIndex = 0`, `fraction = 1`.
 - Cycle 4: `baseIndex = 0`, `fraction = 2`.

- Also verify fraction-only tracking with `phaseInc = 0x200000` using a 2-cycle wait over 3 steps (each step increments `fraction` by exactly 1).

1.8.3 Forward Wrap Done

- **Action:** Set `accumDir = False` (Forward), load `phaseInc = 0x200000`. Set `activeStage = 1` (Attack stage). Run for 2048 cycles to wrap the 32-bit accumulator.
- **Assertion:** Verify that on the 2048th cycle, the register overflows, asserting `segmentDone = True` for exactly 1 cycle.

1.8.4 Reverse Underflow Done

- **Action:** Assert `resetAccum = True` to clear the counter to 0. Set `accumDir = True` (Reverse), load `phaseInc = 0x200000`. Set `activeStage = 4` (Release stage). Enable accumulator.
- **Assertion:** Verify that on the very first cycle, the counter underflows past zero, asserting `segmentDone = True` instantly for 1 cycle.

1.8.5 Decay Target Done Detection

- **Action:** Initialize `accum` to `130 << 24` (`baseIndex = 130`). Set `activeStage = 2` (Decay stage), `sustainLevel = 128`, `accumDir = True` (Reverse), and `phaseInc = 0x200000`. Enable accumulator.
- **Assertion:** Verify that `segmentDone` triggers exactly when `baseIndex` decrements and matches or crosses `<= sustainLevel` (128), and verify that `segmentDone` drops back to `False` on the subsequent cycle when transitioning `activeStage` to 3 (Sustain)..

1.9 Envelope Shaper Unit Test (EnvelopeShaperSim)

Purpose

Verifies the 257-entry LUT curves, shift-add combinational linear interpolation, unipolar-to-bipolar digital scaling, and audio heartbeat gating.

Simulated Environment

- **Component Under Test:** `EnvelopeShaper`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.phaseTick`: Bool (Audio sample gating tick)
- `io.baseIndex`: UInt (8 bits)
- `io.fraction`: UInt (2 bits)
- `io.curveSelect`: UInt (2 bits)
- `io.sustainLevel`: UInt (8 bits)
- `io.activeStage`: UInt (3 bits)
- `io.envelopeOut`: Flow[UInt] (10 bits)
- `io.envelopeOutSigned`: Flow[SInt] (10 bits)

Test Cases

1.9.1 Reset Stability

- **Action:** Assert active-high `reset` for 5 clock cycles while driving active `phaseTick = True` and non-zero inputs.
- **Assertion:** Verify that output flow streams are strictly quiet and zeroed: `envelopeOut.valid = False`, `envelopeOut.payload = 0`, `envelopeOutSigned.valid = False`, and `envelopeOutSigned.payload = 0`.

1.9.2 Multiplierless Shift-Add Accuracy

- **Action:** Set `curveSelect = 00` (Linear). Set `baseIndex = 10` (so `Y0 = 10`, `Y1 = 11`). Drive different values on `fraction`:
 - `fraction = 00`: Assert `envelopeOut.payload = 40` (`Y0` interpolated scale).
 - `fraction = 01`: Assert `envelopeOut.payload = 41` (interpolated `Y0 + 1/4`).
 - `fraction = 10`: Assert `envelopeOut.payload = 42` (interpolated `Y0 + 2/4`).
 - `fraction = 11`: Assert `envelopeOut.payload = 43` (interpolated `Y0 + 3/4`).

1.9.3 Parallel Bipolar Bitwise Scaling

- **Action:** Monitor the relationship between unipolar and bipolar outputs across different phase positions.
- **Assertion:** Verify that `io.envelopeOutSigned.payload` is exactly equal to $(\text{io.envelopeOut.payload} \wedge 0x200) . \text{asSInt}$, confirming that the unipolar range (0 to 1023) maps perfectly to the center-zero bipolar range (-512 to +511) without logic delays or arithmetic units.

1.9.4 Sample Rate Flow Gating Validation

- **Action:** Toggle `phaseTick` between `True` and `False` while checking the output validity.
- **Assertion:** Verify that both `envelopeOut.valid` and `envelopeOutSigned.valid` follow the state of `phaseTick` synchronously with a 1 clock cycle pipeline latency (`RegNext` propagation delay).

2. Integration Tests

This chapter contains the specifications for verifying multi-module and full-system interactive integration behavior.

2.1 Complete System Integration Test (SynthSim)

Purpose

Verifies the end-to-end synthesizer system (`Synth`) for seamless hardware module cooperation, including UART reception, register bank parsing, dynamic digital sound wave synthesis (PWM), volume scaling, and stereo I²S serial streaming in real-time.

Simulated Environment

- **Component Under Test:** `Synth`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.uartRx`: `Bool` (input UART serial receive line)
- `io.i2sBclk`: `Bool` (output I2S bit clock)
- `io.i2sLrc1k`: `Bool` (output I2S left/right channel select)
- `io.i2sData`: `Bool` (output I2S serial audio data stream)

Test Cases

2.1.1 Power-On Silence Idle

- **Action:** Deassert reset and run simulation for 1000 clock cycles with `uartRx` held idle (`True`).
- **Assertion:** Verify that the generated left/right stereo I2S samples are strictly silent (0), confirming the attenuator starts safely muted.

2.1.2 Real-time UART Parameter Modulation

- **Action:** Stream a live byte sequence over the `uartRx` line at 115200 Baud (208 master cycles per bit) to dynamically configure the synthesizer:
 1. Write `0x02` (PWM mode) to Waveform Select (`0x03`).
 2. Write `0x80` (50% Duty cycle) to PWM Width (`0x04`).
 3. Write `0xFF` (Max Volume) to Volume (`0x05`).
 4. Write `0x03` (Enable Envelope + Gate ON) to Envelope Control (`0x40`).
 5. Write `0x00` (Attack = 0) to Envelope Attack (`0x41`).
 6. Write `0x00` (Decay = 0) to Envelope Decay (`0x42`).
 7. Write `0x80` (Sustain = 128) to Envelope Sustain (`0x43`).
 8. Write atomic DDS Frequency tuning word `0x080000` (Low = `0x00`, Mid = `0x00`, High = `0x08` commit to target `15\text{ kHz}`).
- **Assertion:** Capture 25 I2S frames and verify:
 - **Stereo Alignment:** Left and Right samples are perfectly identical for all frames.
 - **Dynamic Audio Response & Envelope Modulation:** Outputs are no longer silent, and sample amplitudes scale dynamically bound under the active ADSR envelope within maximum absolute peaks (≤ 32609), with absolute values scaling up successfully over time (> 5000).

2.2 Complete Envelope Generator Integration Test (EnvelopeGeneratorSim)

Purpose

Verifies the integration of the three submodules, validating the complete 3-cycle pipeline latency, and the delay-matched FSM transition timing.

Simulated Environment

- **Component Under Test:** `EnvelopeGenerator`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.phaseTick`: `Bool`
- `io.syncIn`: `Bool`
- `io.midiClock`: `Bool`
- `io.config`: `EnvelopeConfig`
- `io.envelopeOut`: `Flow[UInt]`
- `io.envelopeOutSigned`: `Flow[SInt]`

Test Cases

2.2.1 Power-On Reset & Boot Stability (Reset Test)

- **Action:** Assert active-high reset for 10 clock cycles while driving `phaseTick = True`, pulsing `syncIn = True`, and setting random non-zero values on the `config` envelope control parameters.
- **Assertion:** Verify that throughout the reset duration and for the 3-cycle pipeline latency window following reset deassertion:
 - Both `envelopeOut.valid` and `envelopeOutSigned.valid` remain strictly `False`.
 - Both unipolar and bipolar payloads are strictly held at 0.

2.2.2 Standard ADSR Envelope Playback (Standard Test)

- **Action:** Configure standard linear ADSR parameters (e.g. `attack = 100`, `decay = 100`, `sustain = 128`, `release = 100`) and drive the playback cycle:
 1. Pulse Gate register Low -> High (ON). Run simulation until FSM hits SUSTAIN.
 2. Pulse Gate register High -> Low (OFF). Run simulation until FSM completes the envelope.
- **Assertion:** Verify the complete envelope profile:
 - **Attack Stage:** The unipolar output climbs monotonically to peak amplitude 1023.
 - **Decay Stage:** The unipolar output decays smoothly to the sustain level of 512 (scaled 0x80).
 - **Sustain Stage:** The output remains locked strictly at 512 with zero cycle-to-cycle amplitude drift.
 - **Release Stage:** The output fades monotonically back to 0.

2.2.3 Pipeline Latency & Sustain Clamping Sync (Standard Test)

- **Action:** Trigger Gate ON at Cycle 0. Drive `phaseTick = True` continuously. Step cycle-by-cycle as FSM transitions from IDLE to ATTACK with `attack = 0` (fastest rise).
- **Assertion:** Verify that:
 - The FSM transitions and resets the accumulator to 0 at **Cycle 2**.
 - The shaper outputs exactly 0 at **Cycle 3** (verifying the 1-cycle register pipeline of the shaper on top of the accumulator register).
 - The output climbs above 0 after exactly **Cycle 50**, proving the high-precision phase accumulator crosses the first LSB scaling threshold correctly.

2.2.4 Simultaneous Gate and Sync Conflict (Edge Case Test)

- **Action:** Pulse both Gate ON and `syncIn` High in the exact same master clock cycle.
- **Assertion:** Hard sync must take absolute priority. The internal accumulator must instantly reset to 0 and FSM must immediately restart in the ATTACK stage (`activeStage = 1`).

2.2.5 Ultra-High Frequency Gate Chattering (Irrational Behavior Test)

- **Action:** Drive the Gate bit rapidly ON -> OFF -> ON -> OFF every 2 clock cycles, simulating extreme physical keyboard bouncing or high-frequency automated trigger chatter.
- **Assertion:** Verify that the envelope generator maintains absolute stability:
 - The state machine transitions safely without entering illegal states.
 - No counter wrapping, deadlock, or phase register leakage occurs.
 - Output values remain strictly bounded within the safe unipolar/bipolar envelopes (0 to 1023) without arithmetic overflow glitches.

2.2.6 Dynamic Mid-Flight Curve Switching (Irrational Behavior Test)

- **Action:** Trigger a standard linear ADSR envelope. Halfway through the ATTACK phase, dynamically change the curve selection register `config.ctrl[6:5]` from Linear (00) to Exponential (01) mid-transition.
- **Assertion:** Verify that the wave shaper accepts the new parameter instantly on the next clock cycle and interpolates smoothly from the current amplitude value using the new exponential ROM table, without causing output signal discontinuities, pops, or wrapping glitches.

Appendix: SpinalSim Best Practices for Reset Verification

This appendix documents the architectural and simulation best practices for implementing and verifying reset stability across all `spinalSynth` modules. It serves as a mandatory guideline for pair-programming, design patterns, and testbench implementations to prevent simulator hangs and race conditions.

A.1 The Verilator Combinational Loop Problem (FSM Gating)

[!NOTE] The code examples in this section are drawn directly from the real-world production implementation of the `EnvelopeCtrl.scala` control module in this repository.

When implementing Finite State Machines (FSMs) or control units, it is common to want control outputs (like `resetAccum` or `runAccum`) to be strictly held inactive (`False` or `0`) during a system reset, even if input trigger signals shift.

The Anti-Pattern (Cyclic Feedback Loop)

A naive approach is to qualify combinational FSM inputs directly with the reset signal:

```
val gateOn = io.config.ctrl(1) && !ClockDomain.current.isResetActive

IDLE.whenIsActive {
  when(gateOn) {
    io.resetAccum := True
    goto(ATTACK)
  }
}
```

- **Why it fails in simulation:** Verilator compiles SpinalHDL's FSM logic into a Verilog state machine. The state register is reset by the `reset` wire. Gating the combinational input paths (like `gateOn`) with the reset signal creates a cyclic dependency graph (`Reset` -> `Inputs` -> `Next State` -> `State Register` -> `Reset`). When manual resets are held active under complex inputs, Verilator's runtime solver enters an **infinite combinational evaluation loop**, hanging the simulation at 100% CPU.

The Best-Practice Pattern (Output-Only Reset Gating)

Always keep the FSM inputs and transitions combinational independent of the reset signal. Instead, use intermediate combinational wires for FSM actions, and apply a **unidirectional output gate** at the top level:

```
// 1. Keep inputs combinational clean
val gateOn = io.config.ctrl(1)

// 2. FSM drives internal outputs
val fsmResetAccum = Bool()
fsmResetAccum := False

val fsm = new StateMachine {
  IDLE.whenIsActive {
    when(gateOn) {
      fsmResetAccum := True
      goto(ATTACK)
    }
  }
}

// 3. Gate the final outputs combinational with reset (Unidirectional)
io.resetAccum := fsmResetAccum && !ClockDomain.current.isResetActive
```

This guarantees that outputs remain strictly quiet during reset without introducing feedback loops into Verilator's next-state logic solver.

A.2 Thread-Safe Reset Verification via forkStimulus Startup

[!NOTE] The verification methodologies documented in this section were developed to address concurrent simulation conflicts during the test of [EnvelopeCtrlSim] (file:///home/moss/Documents/hardware/AI_Audio_HW/spinalSynth/src/test/scala/synth/envelope/EnvelopeCtrlSim.scala).

SpinalSim's `clockDomain.forkStimulus(period)` automatically spawns an asynchronous background thread that drives the master clock and manages the initial power-on reset lifecycle (asserting reset at `$t=0$`, holding it active, and deasserting it after a few nanoseconds).

The Anti-Pattern (Concurrent Thread Collision)

Manually calling `assertReset()` and `deassertReset()` mid-simulation in the main test thread while the background `forkStimulus` thread is running leads to silent overwrites. The background thread will wake up on a timer and deassert the reset pin behind our back, causing stable active-reset assertions to fail.

The Best-Practice Pattern (Startup Window Assertion)

Verify reset stability strictly during **Cycle 1** of the simulation, leveraging the built-in startup reset phase. This is thread-safe, robust, and represents realistic power-on hardware behavior:

```
test("Module unit test - reset & transitions") {
  SimConfig.withWave.compile(new MyModule).doSim { dut =>
    // 1. Initialize clock and let forkStimulus automatically manage reset
    dut.clockDomain.forkStimulus(period = 10)

    // 2. Set safe defaults for all inputs
    dut.io.trigger #= false
    // ...

    // 3. Wait exactly 1 clock cycle to stabilize the simulator delta-cycles
    dut.clockDomain.waitSampling()

    // 4. Assert that outputs are strictly quiet under active startup reset
    assert(!dut.io.controlOut.toBoolean, "Outputs must remain False during active
reset")

    // 5. Let the automatic startup reset complete completely and stabilize
    dut.clockDomain.waitSampling(20)

    // 6. Proceed with normal state transitions...
  }
}
```

A.3 State Transition Synchronization (Delta-Cycle Waiting)

Because simulator input changes (via `#=`) are scheduled combinationally, and `waitSampling()` unblocks precisely on the rising clock edge, FSM register state propagation can sometimes span across a delta-cycle.

- **Rule of Thumb:** Always use a **2-cycle wait** (`waitSampling(2)`) or an additional stabilizing cycle after asserting inputs before verifying next-state registered outputs (like `activeStage`).

- **Pulse Duration Rule:** For edge-sensitive pulses (like `segmentDone`), keep the signal high for **exactly 1 cycle** (`waitSampling()`) and immediately pull it low before waiting for the transition to settle, avoiding double-transitions through multiple states.